# SimEthica-cog-epi: A Framework for Modeling Moral, Cognitive, and Epistemic Dynamics in Agent-Based Systems

David R. Koepsell, J.D., Ph.D.
Department of Philosophy, Texas A&M University
https://github.com/dkoepsell/SimEthica-cog-epi

October 18, 2025

## Contents

**1 Introduction** **3**

**2 Theoretical and Conceptual Background** **4**
2.1 Relational Ontology of Obligation . . . . . 4
2.2 Contradiction Debt and Moral Repair . . . . . 4
2.3 Cognitive Agency and Epistemic Autonomy . . . . . 4
2.4 Moral Energy and Field Dynamics . . . . . 5
2.5 Pluralism as a Structural Condition . . . . . 5

**3 System Architecture** **6**
3.1 Module Overview . . . . . 6
3.2 Execution Workflow . . . . . 6
3.3 Concurrency and Performance . . . . . 7
3.4 Data Logging and Export . . . . . 7
3.5 Extensibility and Modularity . . . . . 7

**4 Agent Design and Dynamics** **8**
4.1 Class Structure . . . . . 8
4.2 Obligation Vector Class . . . . . 10
4.3 Behavioral Loop . . . . . 10
4.4 Trust Dynamics . . . . . 11
4.5 Contradiction Debt Accumulation . . . . . 11
4.6 Agent Visualization . . . . . 12
4.7 Summary of Agent Variables . . . . . 12

**5 Simulation Workflow** **12**
5.1 Initialization Phase . . . 12
5.2 Main Update Cycle . . . 13
5.3 Neighbor Detection and Spatial Partitioning . . . 14
5.4 Metric Update Logic . . . 14
5.5 Steady-State and Termination . . . 14
5.6 Logging and Data Export . . . 14
5.7 Visualization Workflow . . . 15
5.8 Parallel and Batch Execution . . . 16
5.9 Summary . . . 16

**6 Metrics and Observables** **17**
6.1 Overview of Metrics . . . 17
6.2 Moral Energy . . . 17
6.3 Contradiction Debt . . . 17
6.4 Trust Density . . . 18
6.5 Repair Rate . . . 18
6.6 Entropy of Trust . . . 18
6.7 Collapse Index . . . 18
6.8 Correlation and Cross-Metric Analysis . . . 19
6.9 Visualization of Metrics . . . 19
6.10 Interpretive Role of Metrics . . . 19

**7 Usage and Customization** **20**
7.1 System Requirements . . . 20
7.2 Browser Execution . . . 20
7.3 Headless / Batch Execution . . . 21
7.4 Scenario Customization . . . 21
7.5 Modifying `config.js` . . . 21
7.6 Adding New Agent Variables . . . 22
7.7 Defining New Norms . . . 22
7.8 Extending Visualization . . . 22
7.9 Parameter Sweep and Experimentation . . . 23
7.10 File Output and Organization . . . 23
7.11 Reproducibility and Transparency . . . 24

**8 Comparative Framework** **24**
8.1 Comparison Criteria . . . 24
8.2 Comparison with Existing Frameworks . . . 24
8.3 Ontological vs. Instrumental Paradigms . . . 25
8.4 Architecture and Extensibility . . . 25
8.5 Metric Innovation . . . 25
8.6 Transparency and Reproducibility . . . 25

8.7 Performance Considerations . . . . . 25

**9 Applications and Future Work** **25**

9.1 Empirical Testing of Philosophical Hypotheses . . . . . 26
9.2 Integration with Moral Topology Visualization . . . . . 26
9.3 Comparative Regime Analysis . . . . . 26
9.4 Pedagogical and Research Applications . . . . . 27
9.5 Data-Driven Validation and Expansion . . . . . 27
9.6 Technical Enhancements . . . . . 27
9.7 Collaborative and Open Science Outlook . . . . . 27
9.8 Conclusion . . . . . 28

**Abstract**

*SimEthica-cog-epi* is an open-source agent-based simulation platform designed to empirically explore the relational ontology of moral, cognitive, and epistemic systems. The software models agents as moral participants in a dynamic field of obligations, trust, and repair. Each agent is characterized by parameters representing cognitive agency, epistemic autonomy, and moral responsiveness. These parameters evolve through inter-agent interactions governed by a configurable set of norms and obligations, yielding emergent macro-level patterns of moral stability and collapse.

The purpose of this document is to provide a transparent, technical exposition of the systems architecture, data structures, algorithms, and underlying ontological rationale. It is intended for researchers, developers, and philosophers seeking to extend or replicate the simulation framework. In addition to detailing its modular design, this paper situates *SimEthica-cog-epi* in relation to existing agent-based modeling environments (NetLogo, Mesa, Repast, and MASON) and outlines how its metaphysical commitments differ from standard rational-choice or utility-based frameworks.

# 1 Introduction

*SimEthica-cog-epi* is a research-driven software environment that translates philosophical concepts of obligation, trust, repair, and contradiction into a computationally tractable agent-based model. It is part of a broader empirical project designed to test the ontological hypotheses advanced in *The Geometry of the Good*, which posits that pluralism and relational obligation constitute the structural preconditions of stable moral and political systems.

Each agent in the system is represented as a node in a moral network. Agents issue, receive, fulfill, deny, or repair obligations with respect to others, generating an evolving relational topology of moral dependencies. The system measures global indicators such as **moral energy** (fulfilled and repaired obligations), **contradiction debt** (unrepaired moral failures), and **trust density** (aggregate of successful moral exchanges). These metrics provide quantitative insight into the moral health and resilience of simulated societies.

Technically, the platform is implemented in modular JavaScript and built for cross-environment compatibility. Its visualization layer employs the `p5.js` library for real-time rendering, while concurrency is achieved through `Web Workers`. Batch execution is supported through a headless Node.js environment for large-scale data generation. The simulations logging functions produce detailed CSV and JSON files summarizing each generations metrics for reproducible analysis.

The design principles of *SimEthica-cog-epi* emphasize:

- **Transparency**: Full access to source code, configuration, and experiment outputs.
- **Extensibility**: Modular components for agent logic, norm definition, and data export.
- **Philosophical fidelity**: Algorithms grounded in ontological realism rather than instrumental rationality.

In contrast to frameworks such as *NetLogo* or *Mesa*, which are designed for general-purpose modeling, *SimEthica-cog-epi* integrates a moral ontology directly into its data model. Obligations are treated not as choices or strategies, but as formalized relations with ontological weight.

This makes it uniquely suited for exploring metaphysical hypotheses about the structure of ethical life, social order, and epistemic pluralism.

The following sections provide a detailed technical exposition of the system, its theoretical background, and its functional design. By making the full model and its rationale open to inspection, the project aims to bridge computational social simulation, applied ontology, and philosophy of ethics.

# 2 Theoretical and Conceptual Background

The theoretical foundation of *SimEthica-cog-epi* arises from the structural ontology developed in *The Geometry of the Good*. This ontology posits that moral reality consists not of isolated agents with intrinsic moral properties, but of relational fields constituted by directed obligations between participants. The system therefore treats obligation as a formal ontological relation rather than as a subjective moral attitude or instrumental calculation.

## 2.1 Relational Ontology of Obligation

In this framework, an obligation is a directed relation $O(a, b)$ between two agents $a$ and $b$, where $a$ is the obligor and $b$ the obligee. Each obligation has a valence (positive or negative), an expected action, and an associated moral energy cost or benefit. The network of all such directed relations forms a moral field whose global structure determines the health and stability of the simulated society.

This treatment of obligation is inspired by early phenomenological accounts (e.g., Reinach, 1913) and structural realism, wherein social facts are understood as configurations of relations rather than aggregates of individuals. The simulation operationalizes these relations as data structures`ObligationVectors`that are instantiated during agent interactions.

## 2.2 Contradiction Debt and Moral Repair

When an agent fails to fulfill an obligation, the system registers a unit of *contradiction debt* ($CD$), representing a breakdown in the moral structure of the network. Over time, accumulated contradiction debt can reduce trust and destabilize the overall system, analogous to entropy in thermodynamics.

Moral repair is modeled as a directed process by which agents attempt to restore the balance of previously broken obligations. Successful repairs reduce $CD$ and increase trust density, while failed repairs contribute to further instability. The ratio between repair rate and contradiction accumulation serves as a proxy for institutional resilience.

## 2.3 Cognitive Agency and Epistemic Autonomy

Each agent is parameterized by two continuous variables:

- **Cognitive Agency (CA)**: The degree to which the agent autonomously initiates or interprets obligations.

- **Epistemic Autonomy (EA)**: The agents tolerance for divergent perspectives and capacity to maintain trust amid disagreement.

The product of these values influences the probability of successful obligation formation and repair. Systems with high mean CA but low EA tend to display authoritarian or overdetermined dynamics; those with high EA but low CA drift toward anomy or moral collapse. Empirical results from prior runs suggest the existence of a *Critical Curvature Zone*, a range of CAEA alignment that sustains both stability and vitality.

### 2.4 Moral Energy and Field Dynamics

The moral field is quantified through aggregate metrics that reflect the overall systems balance:

$$ME = (F + R) - (D + X) \tag{1}$$

$$TD = \frac{T_s}{N_a} \tag{2}$$

where:

- $F$ = fulfilled obligations,
- $R$ = repaired obligations,
- $D$ = denied obligations,
- $X$ = expired obligations,
- $ME$ = moral energy,
- $T_s$ = total successful trust relations,
- $N_a$ = number of agents.

These global variables evolve over discrete time steps, revealing macro-level patterns of cooperation, decay, or repair within the population.

### 2.5 Pluralism as a Structural Condition

A central theoretical claim embedded in *SimEthica-cog-epi* is that pluralismmaintaining moderate heterogeneity of cognitive and epistemic parametersproduces a stable yet adaptive moral topology. Over-alignment (as in utopian or authoritarian regimes) yields rigidity and collapse, while under-alignment (as in anomic or fragmented systems) prevents the formation of durable trust structures. The model thus enables empirical exploration of pluralism as an ontological rather than purely normative condition.

The next section details how these conceptual structures are implemented in the software architecture.

# 3 System Architecture

The architecture of *SimEthica-cog-epi* is modular, extensible, and designed for both real-time visualization and headless computation. Figure 1 summarizes the systems primary components and their interactions.

## 3.1 Module Overview

- **config.js**: Defines global constants (population size, interaction radius, moral repair rate, etc.) and scenario presets (e.g., pluralist, authoritarian, anomic).
- **agent.js**: Implements the `Agent` class, encapsulating state variables (CA, EA, trust level, debt, position) and behavior methods (move, interact, issueObligation, repair).
- **obligation.js**: Contains the `ObligationVector` class, representing directed moral relations with status tracking (fulfilled, denied, expired, repaired).
- **norms.js**: Defines global rules for obligation formation and interaction logic, including thresholds for trust, proximity, and obligation probability.
- **sketch.js**: Serves as the primary visualization controller using `p5.js`, managing initialization, simulation loop, and rendering.
- **exporter.js**: Handles CSV and JSON data logging at user-specified intervals, supporting reproducible analysis and plotting.
- **scenarios.js**: Provides predefined environmental configurations representing distinct moral or political regimes.

Each module communicates via shared global state objects or through message passing using `Web Workers` for concurrency. The system is compatible with both browser-based execution and Node.js for headless runs.

## 3.2 Execution Workflow

The execution pipeline consists of the following stages:

1. **Initialization**: Load configuration parameters; instantiate agent population; assign random or scenario-specific CA and EA values.
2. **Interaction Loop**: For each time step:
   (a) Agents move according to simple flocking rules (alignment, cohesion, separation).
   (b) Proximate agents may generate obligations based on trust thresholds.
   (c) Obligations are fulfilled, denied, or repaired according to defined probabilities.
   (d) Metrics (moral energy, contradiction debt, trust density) are updated.
3. **Visualization / Logging**: Current state rendered via `p5.js` canvas; metrics written to CSV/JSON logs.

4. **Termination**: Simulation halts after defined generation count or steady-state detection; logs finalized.

### 3.3 Concurrency and Performance

For high agent counts ($N > 500$), the framework uses `Web Workers` to distribute the update loop across multiple threads. Agents state updates are aggregated asynchronously and synchronized each frame. This design enables smooth rendering at 60 FPS with up to several thousand agents, depending on browser and hardware.

### 3.4 Data Logging and Export

All significant variables are recorded each generation for post-processing. A typical CSV output includes:

```
generation,fulfilled,repaired,denied,expired,trust,conflict,debt,
   moral_energy
1,203,18,5,2,0.88,0.12,0.04,0.82
2,211,22,7,3,0.90,0.09,0.03,0.85
...
```

These outputs are used to compute derived metrics such as cumulative repair ratios or moral energy trajectories.

### 3.5 Extensibility and Modularity

Because each behavioral component is isolated in its own module, developers can modify or replace any subsystem independently. For example:

- Introducing a new obligation type requires adding a subclass in `obligation.js` and a corresponding handler in `norms.js`.
- New visualization modes can be defined by extending the `display()` method in `agent.js`.
- Additional global metrics (e.g., entropy of trust distribution) can be registered through the exporter.

This separation of conceptual and operational layers allows the framework to evolve alongside its theoretical goals.

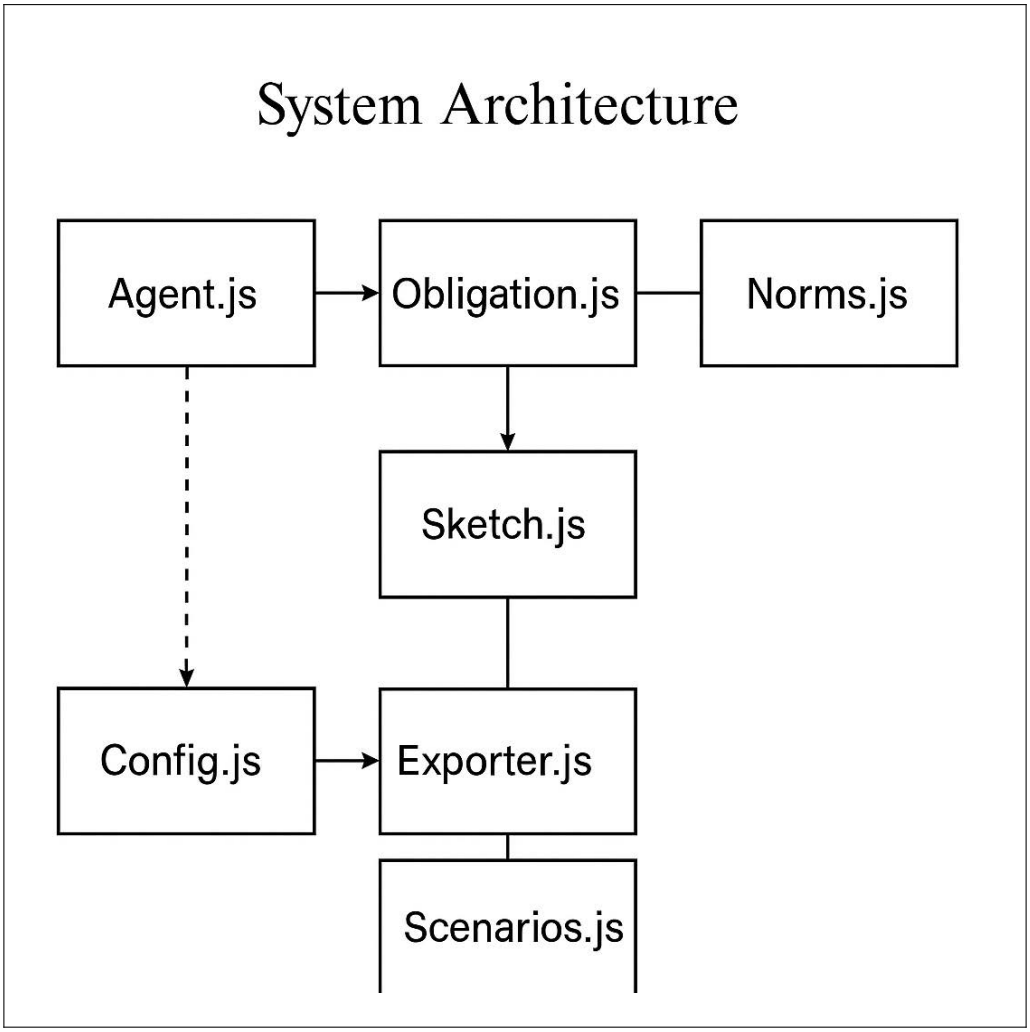


Figure 1: High-level architecture of the *SimEthica-cog-epi* framework showing modular dependencies and data flow.

Subsequent sections describe the agent design, simulation workflow, and data analysis routines in more formal computational detail.

# 4 Agent Design and Dynamics

Agents are the primary ontological and computational entities within *SimEthica-cog-epi*. Each agent represents a locus of moral and epistemic activity: a bearer of obligations, a generator of trust, and a participant in moral repair. The design goal is to minimize anthropomorphic assumptions while maintaining sufficient structure to instantiate relational dynamics consistent with the theoretical ontology.

## 4.1 Class Structure

Each agent is implemented as an instance of the `Agent` class defined in `agent.js`. The class encapsulates the agents persistent state variables, internal memory, and behavior functions executed during each simulation step. The following pseudocode summarizes the class structure:

```
class Agent:
    def __init__(self, id, position, velocity):
        self.id = id
        self.position = position
        self.velocity = velocity

        # Core cognitive and epistemic traits
        self.CA = random.uniform(CA_MIN, CA_MAX)  # Cognitive Agency
        self.EA = random.uniform(EA_MIN, EA_MAX)  # Epistemic Autonomy

        # Dynamic relational variables
        self.trust = 1.0
        self.debt = 0.0
        self.energy = 0.0

        # Memory buffers
        self.obligations_out = []
        self.obligations_in = []
        self.neighbors = []

    def move(self):
        # Basic flocking behavior with boundary wrapping
        self.velocity += alignment(self) + cohesion(self) + separation(
            self)
        self.position = (self.position + self.velocity) % WORLD_SIZE

    def interact(self):
        for n in self.neighbors:
            if random.random() < obligation_probability(self, n):
                self.issue_obligation(n)

    def issue_obligation(self, target):
        ob = ObligationVector(self, target)
        self.obligations_out.append(ob)
        target.obligations_in.append(ob)

    def evaluate_obligations(self):
        for ob in self.obligations_in:
            ob.update_status(self)

    def repair_relations(self):
        for ob in broken_relations(self):
            if random.random() < repair_probability(self):
                ob.repair()
                self.energy += REPAIR_GAIN
            else:
                self.debt += CONTRADICTION_INCREMENT
```

Listing 1: Agent class structure (pseudocode)

The design follows a loosely object-oriented pattern where each agent maintains its own local state and autonomy. State updates occur in parallel during each iteration and are synchronized at the end of the frame.

### 4.2 Obligation Vector Class

Obligations are instantiated as lightweight relational objects linking two agents. Each `ObligationVector` maintains its source and target, its temporal status, and the outcome of its moral transaction.

```
class ObligationVector:
    def __init__(self, source, target):
        self.source = source
        self.target = target
        self.status = 'pending'
        self.age = 0

    def update_status(self, target):
        # Determine outcome based on trust and cognitive factors
        p_fulfill = target.CA * target.EA * target.trust
        p_deny = (1 - target.CA) * (1 - target.EA)
        if random.random() < p_fulfill:
            self.status = 'fulfilled'
        elif random.random() < p_deny:
            self.status = 'denied'
        elif self.age > EXPIRY_LIMIT:
            self.status = 'expired'
        else:
            self.age += 1

    def repair(self):
        self.status = 'repaired'
```

Listing 2: ObligationVector class structure (pseudocode)

The interaction of `Agent` and `ObligationVector` classes defines the basic moral mechanics of the system. Each obligation carries implicit risk and moral potential, reflecting the uncertainty of real-world relational exchanges.

### 4.3 Behavioral Loop

Each simulation step executes a uniform update cycle over the agent population. The behavior loop is expressed as:

1. **Perception:** Each agent identifies its neighbors within the interaction radius $r_i$.

2. **Movement:** The agents position is updated using flocking rules:

$$v_i(t+1) = v_i(t) + \alpha A_i + \beta C_i + \gamma S_i$$

where $A_i$, $C_i$, and $S_i$ represent alignment, cohesion, and separation vectors respectively.

3. **Obligation Generation:** With probability

$$P_{ob} = \eta \cdot \mathrm{CA}_i \cdot \mathrm{EA}_j \cdot T_{ij}$$

an obligation is created between $i$ and $j$, where $T_{ij}$ is trust.

4. **Evaluation:** Pending obligations are resolved as fulfilled, denied, or expired.

5. **Repair:** Agents attempt to repair failed obligations with probability proportional to their repair capacity and moral energy.

6. **Update:** Trust and debt variables are updated according to outcomes:

$$T_{ij}(t+1) = T_{ij}(t) + \delta_F - \delta_D,$$

$$CD_i(t+1) = CD_i(t) + \lambda_D - \lambda_R,$$

where $\delta_F$ and $\delta_D$ are trust adjustments after fulfillment or denial, and $\lambda_D$, $\lambda_R$ are debt increments or reductions.

### 4.4 Trust Dynamics

Trust is modeled as a decaying scalar influenced by the outcome of recent interactions. For each pair $(i, j)$:

$$T_{ij}(t+1) = (1-\rho)T_{ij}(t) + \rho\Delta,$$

where $\rho$ is the update rate and $\Delta$ is the trust delta from the latest obligation. Trust values are clamped between $[0, 1]$. A high trust level increases the likelihood of future obligations and repair cooperation, forming positive feedback loops that can stabilize clusters of agents.

### 4.5 Contradiction Debt Accumulation

Contradiction debt ($CD$) represents the cumulative unfulfilled or denied obligations associated with each agent:

$$CD_i(t+1) = CD_i(t) + \sum_{k \in O_i^-} w_k - \sum_{k \in R_i} r_k,$$

where $O_i^-$ is the set of failed obligations, $R_i$ the set of repairs, $w_k$ the moral weight of each failure, and $r_k$ the magnitude of each repair. High $CD_i$ values reduce an agents capacity to form new obligations, representing moral fatigue or social isolation.

### 4.6 Agent Visualization

The visualization layer renders each agent as a colored node, with color encoding trust level and brightness corresponding to cognitive agency. Obligation vectors are displayed as transient lines connecting agents, fading as they expire or are repaired. This dynamic representation allows researchers to visually identify clusters of trust, breakdowns, or emergent self-organizing moral structures.

### 4.7 Summary of Agent Variables

| Variable | Symbol | Description |
|---|---|---|
| Cognitive Agency | CA | Agents ability to initiate and interpret obligations |
| Epistemic Autonomy | EA | Tolerance for difference; resilience under disagreement |
| Trust | $T_{ij}$ | Dyadic measure of reliability between agents |
| Contradiction Debt | $CD_i$ | Accumulated record of unfulfilled obligations |
| Moral Energy | $ME_i$ | Effective moral vitality after obligations and repairs |
| Repair Rate | $R_i$ | Probability of successful repair attempts |

Table 1: Summary of core agent variables and their functional roles.

Together, these mechanisms generate the emergent macro-structures analyzed in later sections, enabling quantitative study of pluralism, stability, and moral repair in simulated societies.

## 5 Simulation Workflow

The *SimEthica-cog-epi* runtime proceeds through a deterministic yet stochasticized loop of initialization, interaction, evaluation, and logging. This section describes the stepwise workflow from configuration loading to data export, including internal timing logic and example configuration syntax.

### 5.1 Initialization Phase

Upon startup, the main controller (`sketch.js` in browser mode or `main.js` in Node batch mode) executes the following initialization sequence:

1. Load simulation parameters from `config.js`.
2. Initialize random seed for reproducibility (default: `Math.random()` or user-defined).
3. Instantiate $N$ `Agent` objects with randomized positions and velocities.
4. Assign cognitive and epistemic parameters according to the selected scenario:

$$\mathrm{CA}_i \sim \mathcal{N}(\mu_{CA}, \sigma^2_{CA}), \qquad \mathrm{EA}_i \sim \mathcal{N}(\mu_{EA}, \sigma^2_{EA})$$

5. Initialize data loggers and visualization layers.

Example configuration snippet:

```
const SIM_CONFIG = {
  NUM_AGENTS: 800,
  INTERACTION_RADIUS: 45,
  MAX_SPEED: 2.0,
  CA_MEAN: 0.7,
  CA_SD: 0.15,
  EA_MEAN: 0.7,
  EA_SD: 0.10,
  REPAIR_RATE: 0.08,
  TRUST_DECAY: 0.02,
  MAX_GENERATIONS: 3000,
  LOG_INTERVAL: 10,
  SCENARIO: "pluralist"
};
```

Listing 3: Configuration parameters in config.js

These parameters define the initial conditions for the agent population and control the runtime duration, interaction density, and granularity of exported data.

### 5.2 Main Update Cycle

Each generation (or frame) of the simulation executes the following ordered steps, expressed in pseudocode for clarity:

```
for generation in range(MAX_GENERATIONS):

    # 1. Neighbor detection
    for agent in agents:
        agent.neighbors = find_neighbors(agent, INTERACTION_RADIUS)

    # 2. Movement update
    for agent in agents:
        agent.move()

    # 3. Obligation generation
    for agent in agents:
        agent.interact()

    # 4. Obligation evaluation
    for agent in agents:
        agent.evaluate_obligations()

    # 5. Moral repair phase
    for agent in agents:
        agent.repair_relations()

    # 6. Metrics aggregation
```

```
    global_metrics.update(agents)

    # 7. Visualization and logging
    if generation % LOG_INTERVAL == 0:
        exporter.record(generation, global_metrics)

    # 8. Termination condition
    if check_steady_state(global_metrics):
        break
```

Listing 4: Simulation main loop (simplified pseudocode)

### 5.3 Neighbor Detection and Spatial Partitioning

To optimize proximity checks, the framework uses a uniform grid spatial hash. Each cell maintains a list of agents within its bounds; only adjacent cells are scanned per frame. This reduces neighbor search complexity from $O(N^2)$ to approximately $O(N)$ for sparse populations. The algorithm can be optionally replaced by a quadtree implementation for higher spatial fidelity.

### 5.4 Metric Update Logic

At each generation, the system computes global and local metrics as:

$$ME(t) = (F(t) + R(t)) - (D(t) + X(t))$$

$$TD(t) = \frac{\sum_{i=1}^{N} \text{trust}_i(t)}{N}$$

$$CD(t) = \frac{\sum_{i=1}^{N} \text{debt}_i(t)}{N}$$

These values are logged in real time and appended to the output dataset. Cumulative values can be derived offline for longitudinal analysis.

### 5.5 Steady-State and Termination

A simulation may terminate before reaching the maximum generation count if it satisfies a steady-state condition defined as:

$$|\Delta ME_t| < \epsilon_M \quad \wedge \quad |\Delta TD_t| < \epsilon_T$$

for a user-defined tolerance $\epsilon$ over $k$ successive frames (default: $\epsilon_M = 0.001$, $k = 50$). Alternatively, manual termination is available via the interface or command line.

### 5.6 Logging and Data Export

The exporter module records both global and per-agent data. A typical CSV file includes:

```
generation,fulfilled,repaired,denied,expired,trust,debt,moral_energy,
    entropy
```

```
1,210,18,5,3,0.89,0.06,0.83,0.12
2,225,19,7,2,0.91,0.05,0.86,0.11
...
```

In batch mode, each run is automatically timestamped and saved to the `/logs` directory with a naming convention:

agentLog_scenario_YYYY-MM-DDTHH-MM-SS.csv

The JSON logger mirrors this data for programmatic use in Python, R, or JavaScript-based analytics pipelines.

### 5.7 Visualization Workflow

In browser mode, the visualization loop (handled by `p5.js`) updates at a target frame rate of 60 FPS. Each agent is rendered as a node whose color encodes trust, and connecting lines represent active obligations. Figure 2 summarizes the simulation pipeline.

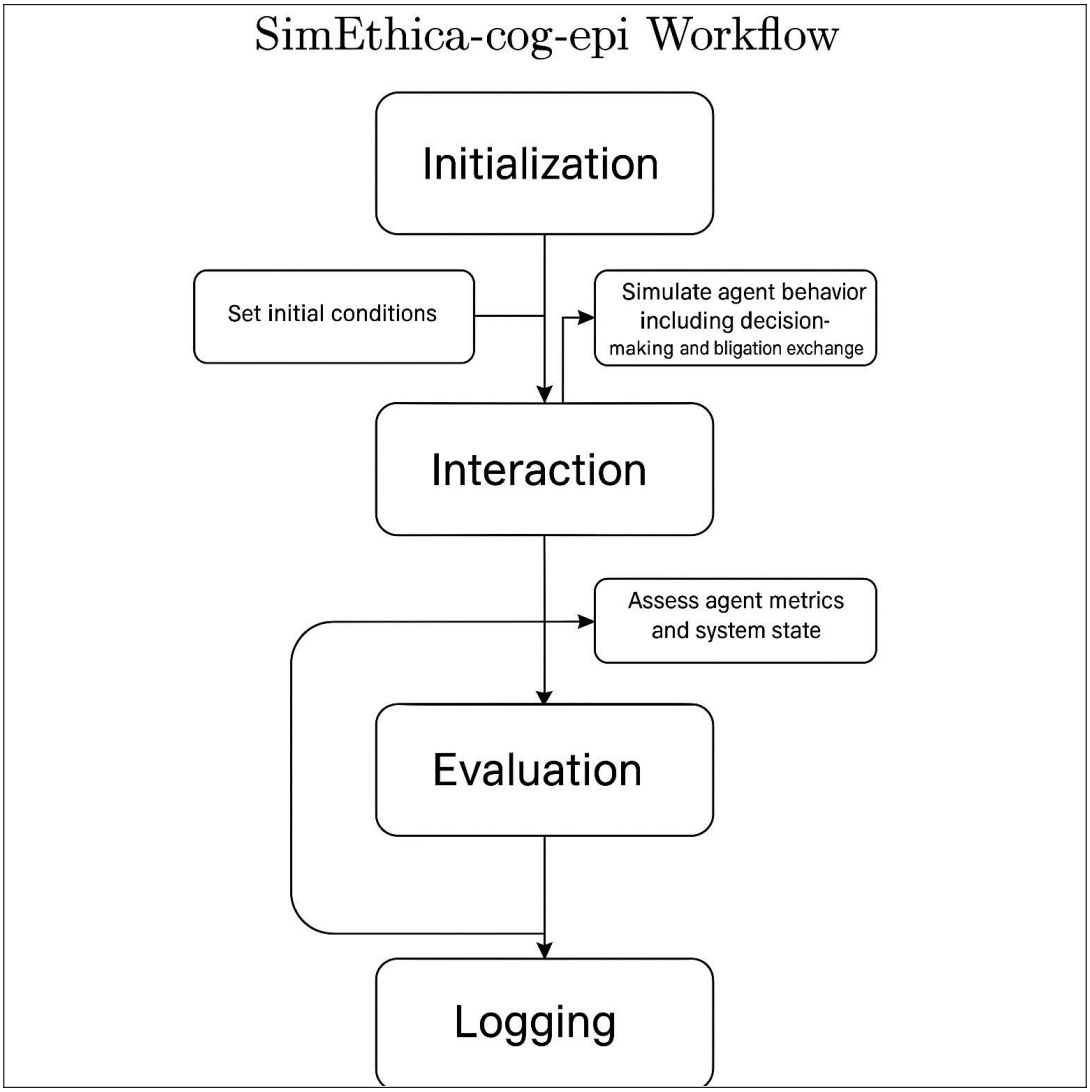


Figure 2: Overview of the *SimEthica-cog-epi* simulation workflow, showing initialization, interaction, evaluation, and logging phases.

### 5.8 Parallel and Batch Execution

For experimental reproducibility, the system supports headless execution using Node.js. Each run can be configured with a scenario file specifying the number of iterations and random seed. Example command-line invocation:

```
node main.js --scenario pluralist --seed 12345 --runs 10 --log ./logs/
```

Listing 5: Batch-mode execution example

This enables large-scale parameter sweeps across different regimes, such as pluralist vs. authoritarian systems, for statistical analysis of emergent stability patterns.

### 5.9 Summary

The simulation workflow is fully deterministic at the level of algorithmic control but stochastic in local interactions. Every frame constitutes a microcosm of moral dynamics whose aggregate

evolution yields the macroscopic metrics analyzed in later sections. This structure allows both theoretical replication of moral field hypotheses and empirical evaluation through repeatable computational experiments.

# 6 Metrics and Observables

The analytical core of *SimEthica-cog-epi* consists of a suite of metrics that capture the evolving moral, cognitive, and epistemic state of the simulated society. These observables are computed every generation and exported to CSV/JSON for external analysis. Each metric corresponds to a theoretically significant ontological construct and is defined by explicit mathematical functions.

## 6.1 Overview of Metrics

Table 2 summarizes the principal system-level observables tracked during simulation runs.

| **Metric** | **Symbol** | **Definition / Purpose** |
|---|---|---|
| Fulfilled Obligations | $F_t$ | Total obligations successfully honored in generation $t$ |
| Repaired Obligations | $R_t$ | Total obligations restored through repair attempts |
| Denied Obligations | $D_t$ | Obligations explicitly rejected by recipients |
| Expired Obligations | $X_t$ | Obligations unresolved within time limit |
| Moral Energy | $ME_t$ | System vitality $(F_t + R_t) - (D_t + X_t)$ |
| Contradiction Debt | $CD_t$ | Mean accumulated unfulfilled obligations |
| Trust Density | $TD_t$ | Average inter-agent trust value |
| Repair Rate | $RR_t$ | Ratio of successful repairs to total failures |
| Entropy of Trust | $H_t$ | Dispersion of trust distribution across agents |
| Collapse Index | $C_t$ | Normalized decline in $ME_t$ and $TD_t$ over $\Delta t$ |

Table 2: Core observables computed by the simulation each generation.

## 6.2 Moral Energy

Moral energy quantifies the overall balance between constructive and destructive moral events within the system:

$$ME_t = (F_t + R_t) - (D_t + X_t)$$

Positive $ME_t$ values indicate net coherence and moral vitality, whereas sustained negative values denote disintegration or collapse. For normalized comparisons across populations of differing size $N$, the system computes:

$$ME_t^* = \frac{ME_t}{N}$$

## 6.3 Contradiction Debt

Contradiction debt ($CD_t$) represents the cumulative moral deficit of the system, aggregating all unfulfilled obligations and failed repairs:

$$CD_t = \frac{1}{N} \sum_{i=1}^{N} CD_i(t)$$

At the agent level:

$$CD_i(t+1) = CD_i(t) + \lambda_D(D_i + X_i) - \lambda_R R_i$$

where $\lambda_D$ and $\lambda_R$ are scalar weight factors controlling the rate of accumulation and reduction. A high mean $CD_t$ correlates with systemic brittleness and declining trust density.

### 6.4 Trust Density

Trust density ($TD_t$) measures the global average of dyadic trust relations:

$$TD_t = \frac{1}{N}\sum_{i=1}^{N} \overline{T_i}(t)$$

where $\overline{T_i}$ is the mean of all trust values held by agent $i$. Empirically, $TD_t$ tracks network cohesion and serves as a leading indicator of either stabilization (rising $TD_t$) or fragmentation (declining $TD_t$).

### 6.5 Repair Rate

The repair rate expresses the systems restorative capacity:

$$RR_t = \frac{R_t}{R_t + D_t + X_t}$$

This metric indicates the proportion of moral breakdowns that are successfully repaired. In resilient pluralist regimes, $RR_t$ tends to stabilize above 0.5; in collapsed or authoritarian conditions, it approaches zero.

### 6.6 Entropy of Trust

To capture heterogeneity in trust distribution, the simulation computes Shannon entropy across all trust values:

$$H_t = -\sum_{b=1}^{B} p_b \log(p_b)$$

where $p_b$ is the probability of trust values falling within bin $b$ over the range $[0, 1]$. High $H_t$ denotes epistemic diversity and distributed trust, while low $H_t$ indicates clustering and potential homogenization. This measure provides a quantitative indicator of pluralism.

### 6.7 Collapse Index

The collapse index is a composite measure reflecting the rate of decline in both moral energy and trust density over a time window $\Delta t$:

$$C_t = 1 - \frac{ME_{t+\Delta t} + TD_{t+\Delta t}}{ME_t + TD_t}$$

Values $C_t > 0.2$ indicate accelerated moral or epistemic decay; $C_t \approx 0$ corresponds to equilibrium. This index serves as a diagnostic signal for the onset of systemic rupture or moral

fatigue.

### 6.8 Correlation and Cross-Metric Analysis

For each run, *SimEthica-cog-epi* computes inter-metric correlations to identify emergent dependencies:

$$\rho(ME, TD) = \frac{\mathrm{cov}(ME, TD)}{\sigma_{ME}\sigma_{TD}}$$

A high positive correlation between $ME$ and $TD$ is typically associated with stable pluralist configurations. Negative or oscillatory correlations are symptomatic of instability or collapse cycles.

### 6.9 Visualization of Metrics

The exported data allow real-time and post-run visualization. Typical analysis includes:

- Plotting $ME_t$, $TD_t$, and $CD_t$ over generations to identify steady states or phase transitions.
- Heatmaps of trust density versus cognitive/epistemic means $(\mu_{CA}, \mu_{EA})$ to reveal the Critical Curvature Zone.
- 3D moral topologies mapping $ME$ as a surface over $(CA, EA)$ space.

Example plotting command (Python, using `matplotlib`):

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("agentLog_pluralist_2025-10-08.csv")
plt.plot(data["generation"], data["moral_energy"], label="Moral Energy"
    )
plt.plot(data["generation"], data["trust"], label="Trust Density")
plt.plot(data["generation"], data["debt"], label="Contradiction Debt")
plt.legend()
plt.xlabel("Generation")
plt.ylabel("Metric Value")
plt.title("System Dynamics: Pluralist Regime")
plt.show()
```

Listing 6: Example post-processing visualization

### 6.10 Interpretive Role of Metrics

Each metric not only quantifies performance but also corresponds to a theoretical construct within the broader ontological framework:

- **Moral Energy** → vitality of relational structure.
- **Contradiction Debt** → accumulation of unrepairable contradictions.

- **Trust Density** → emergent cohesion through mutual recognition.
- **Repair Rate** → institutional and social elasticity.
- **Entropy of Trust** → pluralism as structural diversity.

These observables form the empirical bridge between the metaphysical commitments of the model and its computational behavior. They enable quantitative testing of the central hypothesis: that pluralist systemsthose maintaining moderate variance in cognitive and epistemic parametersmaximize moral energy while minimizing contradiction debt.
The next section outlines practical usage, configuration, and customization options for running and modifying the simulation.

# 7 Usage and Customization

*SimEthica-cog-epi* is designed for full transparency and ease of modification. Researchers can run it interactively in the browser or execute batch experiments in headless Node.js mode. This section provides instructions for installation, configuration, scenario design, and code extension.

## 7.1 System Requirements

- Node.js v18 or later
- Modern browser supporting `WebGL` and `ES6`
- Optional: Python 3.8 or later for post-processing (e.g., `pandas`, `matplotlib`)

Clone the repository:

```
git clone https://github.com/dkoepsell/SimEthica-cog-epi.git
cd SimEthica-cog-epi
```

Listing 7: Repository clone command

## 7.2 Browser Execution

For interactive visual runs, open `index.html` in any compatible browser. The UI presents basic controls for start, pause, reset, and parameter display. Real-time metrics appear in the console and on-screen overlays.

```
npm install   # installs dependencies
npm run serve # hosts local webserver (optional)
# Then open: http://localhost:8080/
```

Listing 8: Launching in browser mode

Users can toggle visualization features such as obligation vectors, trust lines, or agent color encoding via the GUI or direct edits in `sketch.js`.

### 7.3 Headless / Batch Execution

Batch mode is preferred for reproducibility and large-scale parameter sweeps. The command below executes 10 runs of the pluralist scenario and stores logs under `/logs/`.

```
node main.js --scenario pluralist --runs 10 --seed 12345 --log ./logs/
```

Listing 9: Batch mode with scenario and random seed

Arguments:

- `--scenario`: selects the configuration preset from `scenarios.js`
- `--runs`: number of independent simulations
- `--seed`: fixed random seed for determinism
- `--log`: output directory for CSV / JSON results

### 7.4 Scenario Customization

All predefined scenarios (pluralist, authoritarian, anomic, utopian, collapsed) reside in `scenarios.js`. Each defines default parameters for cognitive and epistemic distributions and moral repair rate. Users can duplicate and modify entries to create new environments.

```
const SCENARIOS = {
  "pluralist": { CA_MEAN: 0.70, EA_MEAN: 0.70, REPAIR_RATE: 0.08 },
  "authoritarian": { CA_MEAN: 0.90, EA_MEAN: 0.20, REPAIR_RATE: 0.03 },
  "experimental": { CA_MEAN: 0.65, EA_MEAN: 0.50, REPAIR_RATE: 0.10 }
};
```

Listing 10: Example of a custom scenario block

Adjusting these parameters directly affects the shape of the moral field and resulting dynamics.

### 7.5 Modifying `config.js`

The file `config.js` contains global runtime settings accessible to all modules. Typical parameters include population size, interaction radius, trust decay, repair rate, and logging interval.

```
const SIM_CONFIG = {
  NUM_AGENTS: 1000,
  INTERACTION_RADIUS: 40,
  REPAIR_RATE: 0.08,
  TRUST_DECAY: 0.02,
  LOG_INTERVAL: 10,
  MAX_GENERATIONS: 3000
};
```

Listing 11: Key configuration variables

Changing any of these values allows experimentation with population density, moral responsiveness, or timescale.

### 7.6 Adding New Agent Variables

Developers can easily introduce new agent-level properties such as `empathy`, `influence`, or `resilience`. Steps:

1. Declare the variable in the `Agent` constructor.
2. Define its update logic in `agent.js` (within `interact()` or `repair_relations()`).
3. Include the variable in the exporter output for analysis.

```
this.empathy = random(0.0, 1.0);

// Example influence on repair probability
function repair_probability(agent) {
  return BASE_REPAIR_RATE * agent.empathy * agent.trust;
}
```

Listing 12: Adding empathy as a new variable

### 7.7 Defining New Norms

Normative logic governing obligation creation and evaluation resides in `norms.js`. To add new behavioral rules:

- Define a function `applyNorm(agentA, agentB)` returning a boolean for obligation formation.
- Register it in the global norm set used by the interaction loop.

```
function empathyNorm(a, b) {
  return (a.empathy + b.empathy) / 2 > 0.6 && distance(a, b) <
      INTERACTION_RADIUS;
}
NORMS.push(empathyNorm);
```

Listing 13: Example of adding a new obligation norm

### 7.8 Extending Visualization

Visualization is modular and controlled by the `display()` methods in `agent.js` and `sketch.js`. Developers may:

- Modify color mapping (e.g., color by debt, energy, or empathy).
- Add overlays for system metrics.
- Toggle drawing of obligation vectors.

Example: rendering moral debt intensity.

```
let d = constrain(agent.debt, 0, 1);
fill(255 * d, 50, 50); // red hue for moral fatigue
ellipse(agent.position.x, agent.position.y, 5, 5);
```

Listing 14: Color coding by contradiction debt

### 7.9 Parameter Sweep and Experimentation

To explore parameter sensitivity, batch scripts can be used to iterate over ranges of CA, EA, or repair rate values. Example Python pseudocode:

```
import os, itertools, subprocess
CAs = [0.6, 0.7, 0.8]
EAs = [0.4, 0.6, 0.8]
for ca, ea in itertools.product(CAs, EAs):
    subprocess.run([
        "node", "main.js",
        "--scenario", "custom",
        "--CA_MEAN", str(ca),
        "--EA_MEAN", str(ea)
    ])
```

Listing 15: Parameter sweep driver (Python)

The resulting logs can be aggregated to construct contour plots of system stability across the CAEA parameter space.

### 7.10 File Output and Organization

By default, the repository structure is as follows:

```
SimEthica-cog-epi/
  agent.js
  obligation.js
  norms.js
  config.js
  sketch.js
  exporter.js
  scenarios.js
  logs/
  index.html
  main.js
  docs/
```

Researchers are encouraged to store derived data, plots, and LaTeX reports under `/docs` for complete transparency.

### 7.11 Reproducibility and Transparency

All random number generation can be seeded via the `--seed` argument to enable deterministic runs. The exported datasets include metadata (simulation ID, timestamp, configuration hash) ensuring that results can be precisely reproduced. The model, code, and documentation are released under an open license, allowing peer verification of results and theoretical claims.
The next section situates *SimEthica-cog-epi* within the broader landscape of agent-based modeling environments and outlines its distinctive philosophical and technical orientation.

## 8 Comparative Framework

*SimEthica-cog-epi* occupies an unusual position among agent-based modeling (ABM) platforms. Whereas traditional frameworks are designed for empirical or economic modeling, this system serves as a research instrument for testing ontological and ethical hypotheses. Its purpose is not prediction or optimization but the exploration of formal structures governing relational, cognitive, and epistemic order. This section contrasts its architecture and theoretical orientation with representative ABM systems.

### 8.1 Comparison Criteria

For comparison, we evaluate across six dimensions:

1. **Philosophical Orientation** what theoretical commitments inform the model.
2. **Modeling Paradigm** how agents are represented and interact.
3. **Extensibility** ease of modification and openness of architecture.
4. **Computational Framework** programming language, concurrency, and visualization.
5. **Metric Design** which system observables are prioritized.
6. **Transparency** availability of code, documentation, and interpretability.

### 8.2 Comparison with Existing Frameworks

Table 3 summarizes the main distinctions between *SimEthica-cog-epi* and leading ABM environments.

| Framework | Language | Orientation | Use Case |
|---|---|---|---|
| NetLogo | Java / Logo | Educational / empirical | Social / ecological model |
| Mesa | Python | Research / extensible | Economic, epidemiologic |
| Repast | Java | Enterprise-grade ABM | Social science simulation |
| MASON | Java | High-performance ABM toolkit | Multi-agent systems |
| FLAME GPU | C++ / CUDA | GPU-parallel ABM | Swarm / biological simul |
| **SimEthica-cog-epi** | JavaScript (p5.js / Node) | Ontological-ethical modeling | Moral, cognitive, episten |

Table 3: Comparison of *SimEthica-cog-epi* with major ABM frameworks.

### 8.3 Ontological vs. Instrumental Paradigms

Conventional agent-based frameworks treat agents as utility-maximizing decision makers or adaptive entities responding to environmental incentives. *SimEthica-cog-epi*, by contrast, models agents as participants in a moral field, where actions are not chosen to optimize payoff but to maintain or repair ontological consistency. The governing dynamics emerge from relational necessity rather than from game-theoretic strategy. This represents a categorical shift from instrumental to structural modeling.

### 8.4 Architecture and Extensibility

Technically, *SimEthica-cog-epi* is closer in modularity to Mesa or MASON but with lighter resource overhead. Its JavaScript implementation allows real-time visualization and concurrent computation through `Web Workers`, achieving interactivity absent in Python-based systems. Unlike monolithic GUIs such as NetLogo, the source code exposes every functional layerfrom configuration to renderingmaking it suitable for direct philosophical or computational modification.

### 8.5 Metric Innovation

Where typical ABM environments output population counts, mean fitness, or infection rates, this framework records relational metricsmoral energy, contradiction debt, trust density, and entropy of trust. These observables have no analogue in traditional ABM systems and were designed to correspond directly to metaphysical concepts of obligation, recognition, and repair. This metric design enables empirical study of philosophical propositions such as the Critical Curvature Zone, where systems maintain maximal vitality at intermediate heterogeneity.

### 8.6 Transparency and Reproducibility

All source files, configuration presets, and datasets are public under an open-access license. No compiled binaries or opaque data structures obscure the logic of simulation. Every numerical result can be regenerated from scriptable configuration files, allowing peer review not only of outcomes but of philosophical assumptions. This transparency contrasts with many ABM toolkits that encapsulate behavior within compiled classes or GUI components.

### 8.7 Performance Considerations

While not optimized for GPU-scale computation like FLAME GPU, the JavaScript/WebGL architecture is adequate for thousands of

## 9 Applications and Future Work

*SimEthica-cog-epi* was conceived not merely as a simulation platform but as an experimental instrument linking ontological theory with computational verification. Its applications span empirical philosophy, social modeling, and educational technology.

## 9.1 Empirical Testing of Philosophical Hypotheses

The primary use of *SimEthica-cog-epi* is the empirical evaluation of ontological hypotheses derived from the relational realism articulated in *The Geometry of the Good*. By encoding moral relations, trust, and repair as measurable quantities, the simulation allows philosophical propositions to be treated as falsifiable. For example:

- The hypothesis that moderate cognitiveepistemic heterogeneity maximizes moral energy and minimizes contradiction debt.
- The claim that authoritarian systems yield short-term stability but long-term moral exhaustion.
- The expectation that high repair capacity is a necessary but insufficient condition for systemic resilience.

These hypotheses can be quantitatively tested through controlled parameter sweeps and analyzed via exported metrics. Each run provides data suitable for regression, clustering, or network analysis, supporting empirical philosophy through computational reproducibility.

## 9.2 Integration with Moral Topology Visualization

The framework integrates directly with external moral topology visualizers that map system-wide metricssuch as moral energy or trust densityover the cognitiveepistemic plane $(CA, EA)$. Generated CSV outputs can be fed into 3D surface-plot modules (e.g., using `Three.js` or `Plotly`) to reveal the emergent curvature zones described theoretically in prior work. This provides an interactive empirical analogue to conceptual moral geometry.

## 9.3 Comparative Regime Analysis

*SimEthica-cog-epi* supports scenario modeling of distinct moralpolitical regimes, such as:

- **Pluralist Regime:** High EA variance, balanced CA; stable moral energy and distributed trust.
- **Authoritarian Regime:** High CA, low EA; rigid coherence followed by collapse.
- **Anomic Regime:** Low CA, high EA variance; fragmentation and moral entropy.
- **Collapsed Regime:** Low values for both CA and EA; minimal obligations, maximal contradiction debt.

Comparative data across these regimes contribute to ongoing analyses of social stability, repair capacity, and the structural ontology of law and democracy.

## 9.4 Pedagogical and Research Applications

In academic contexts, *SimEthica-cog-epi* serves as a bridge between philosophical theory and computational practice. It enables students in philosophy of law, ethics, and ontology to visualize abstract structures as dynamic systems. Assignments can involve altering norms, defining new moral metrics, or comparing simulation outputs with theoretical predictions.

For research teams, the model provides an extensible base for collaborative experimentsallowing integration with empirical datasets, network topology generators, or reinforcement-learning agents representing institutional actors.

## 9.5 Data-Driven Validation and Expansion

Future versions will include:

- Import pipelines for empirical data (e.g., governance, trust indices, social survey metrics) to correlate real-world observations with simulated moral fields.
- Automatic calibration of agent parameters to fit historical case data (e.g., collapse or stabilization events).
- Integration with ontology-based data stores using Basic Formal Ontology (BFO) classes for formal semantic traceability.

These additions will enable both inductive and deductive validation of the models theoretical claims.

## 9.6 Technical Enhancements

Planned technical developments include:

- GPU acceleration for large-scale populations using WebGPU or CUDA bindings.
- Distributed execution across multiple nodes for extended simulations of $N > 10^5$.
- Enhanced visualization with interactive dashboards and real-time analytics.
- Modular plug-ins for new moral dimensions such as empathy, institutional authority, or resource exchange.

## 9.7 Collaborative and Open Science Outlook

The full source, documentation, and results are hosted openly on GitHub to encourage replication and peer collaboration. Researchers are invited to fork, modify, and extend the codebase, contributing additional scenarios, metrics, or theoretical models. Every simulation, parameter set, and result can be versioned and cited to promote transparent, data-driven philosophy.

### 9.8 Conclusion

*SimEthica-cog-epi* operationalizes moral and epistemic ontology as a reproducible computational framework. It demonstrates that structural philosophical conceptsobligation, trust, contradiction, repair, and pluralismcan be rendered into formal systems and subjected to empirical scrutiny. Beyond a research tool, it represents a proof of concept: that metaphysical realism and computational modeling are not mutually exclusive but mutually reinforcing. The ongoing development of this framework will continue to merge philosophical rigor with open scientific method, creating a shared experimental platform for the study of relational ethics and social ontology.

## References


[1] Koepsell, D. R. (2024). *The Geometry of the Good.* (forthcoming manuscript).

[2] Arp, R., Smith, B., and Spear, A. D. (2015). *Building Ontologies with Basic Formal Ontology.* MIT Press.

[3] Gilbert, M. (1990). *Walking Together: A Paradigmatic Social Phenomenon.* Midwest Studies in Philosophy, 15(1), 114.

[4] Axelrod, R. (1997). *The Complexity of Cooperation.* Princeton University Press.

[5] Railsback, S. F., and Grimm, V. (2019). *Agent-Based and Individual-Based Modeling.* Princeton University Press.